DS	**Session**	Tree 2	**Question Information**	Level 1 Challenge 76

Question description

Your task is to maintain an array of n values and efficiently process the following types of queries:

1. Increase the first value in range [a,b] by 1, the second value by 2, the third value by 3, and so on.
2. Calculate the sum of values in range [a,b].

Input

The first input line has two integers n and q: the size of the array and the number of queries.

The next line has n values t1,t2,…,tn: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is either "1 a b" or "2 a b".

Output

Print the answer to each sum query.

Constraints

- $1 \le n,q \le 2 \cdot 10^5$
- $1 \le t_i \le 10^6$
- $1 \le a \le b \le n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3
3 7 2 2 5
3 2 3
1 3 6
1 2 3
```

EXPECTED OUTPUT

```
9
```

Test Case 2

INPUT (STDIN)

```
5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5
```

EXPECTED OUTPUT

```
17
32
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void build(int k,int
l,int r)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
2
```

Test Case 2

TOKEN COUNT

```
800
```

Test Case 3

NLOC

```
71
```

Code

range_poly_queries.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXN 200005

long long tree[4 * MAXN];
long long lazy_a[4 * MAXN];
long long lazy_d[4 * MAXN];
long long arr[MAXN];

void build(int k,int l,int r) {
    lazy_a[k] = 0;
    lazy_d[k] = 0;
    if (l == r) {
        tree[k] = arr[l];
        return;
    }
    int mid = (l + r) / 2;
    build(2 * k, l, mid);
    build(2 * k + 1, mid + 1, r);
    tree[k] = tree[2 * k] + tree[2 * k + 1];
}

void apply_ap(int k, int l, int r, long long a, long long d) {
    long long len = r - l + 1;
    tree[k] += a * len + d * len * (len - 1) / 2;
    lazy_a[k] += a;
    lazy_d[k] += d;
}

void push(int k, int l, int r) {
    if (lazy_a[k] == 0 && lazy_d[k] == 0) return;
    int mid = (l + r) / 2;
    apply_ap(2 * k, l, mid, lazy_a[k], lazy_d[k]);
    long long right_a = lazy_a[k] + lazy_d[k] * (mid - l + 1);
    apply_ap(2 * k + 1, mid + 1, r, right_a, lazy_d[k]);
    lazy_a[k] = 0;
    lazy_d[k] = 0;
}

void update_range(int k, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) {
        long long a = l - ql + 1;
        apply_ap(k, l, r, a, 1);
        return;
    }
    push(k, l, r);
    int mid = (l + r) / 2;
    if (ql <= mid) update_range(2 * k, l, mid, ql, qr);
    if (qr > mid) update_range(2 * k + 1, mid + 1, r, ql, qr);
    tree[k] = tree[2 * k] + tree[2 * k + 1];
}

long long query_range(int k, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tree[k];
    push(k, l, r);
    int mid = (l + r) / 2;
    long long res = 0;
    if (ql <= mid) res += query_range(2 * k, l, mid, ql, qr);
    if (qr > mid) res += query_range(2 * k + 1, mid + 1, r, ql, qr);
    return res;
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;
    for (int i = 1; i <= n; i++) scanf("%lld", &arr[i]);
    build(1, 1, n);
    while (q--) {
        int type, a, b;
        scanf("%d %d %d", &type, &a, &b);
        if (type == 1) {
            update_range(1, 1, n, a, b);
        } else {
            printf("%lld\n", query_range(1, 1, n, a, b));
        }
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc range_poly_queries.c -o range_poly_queries
PS C:\C Programming> .\range_poly_queries
5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5
17
32
PS C:\C Programming>